SYSTEM AUDIT REPORT — Transportation Management Portal (TMP)

Audit date: 2026-10-01 **Branch audited:** `refactor/structure-hardening` (HEAD `c3274ca` + uncommitted working tree, incl. untracked `lib/actions/`, `lib/cache-tags.ts`, `lib/queries/fleet.ts`, `lib/routing/corridor-core.ts`, migration `20261001124637_add_query_indexes`) **Stack:** Next.js 16 (App Router) · React 19 · TypeScript (strict) · Prisma 7 + Neon Postgres · jose JWT sessions · nodemailer · OSRM + Nominatim **Scope:** `app/`, `actions/`, `components/`, `lib/` (excl. `lib/generated`), `hooks/`, `proxy.ts`, `prisma/schema.prisma` — 213 TS/TSX files, 39 pages, 20 action files (77 exported Server Actions). **Mode:** Read-only. No source file was modified; this report is the only file written.

---

0. Executive Summary

Pillar	Grade	One-line verdict
1. Architecture & Folder Structure	**B−**	Clear top-level domain split, but pages bypass `lib/queries`, read-logic lives in `actions/`, and role areas are copy-pasted.
2. System Design & Routing Logic	**C+**	Good direction (pure `side-selection`, new sync `corridor-core`), but `corridor-core.ts` is an **unwired duplicate**, and I/O + decision logic are still fused in a 1,596-line orchestrator.
3. Performance & Scalability	**C**	Serial OSRM calls in nested loops, no fetch timeouts, full-history scans per run, render-time DB writes, and a cache layer that is **declared but never invalidated**.
4. Security & Best Practices	**C+**	Guard-first RBAC and error sanitization are excellent (73/73), **but** password hashes/reset tokens leak to the browser, Facilities can escalate/take over accounts, and there is no audit log.
5. Code Quality & Technical Debt	**B**	`tsc` 0 errors, zero `any` / `@ts-ignore`; but `eslint` reports **7 errors / 9 warnings**, ~8 orphaned components, 8 uncalled-but-reachable Server Actions, and no test script.

Top 10 fixes, in priority order

#	Severity	Issue	Where
1	**Critical**	Full `User` rows (incl. `passwordHash`, `passwordResetToken`) serialized to client components / returned from actions	`app/operations/adhoc/page.tsx:10-16,46`, `app/facilities/adhoc/page.tsx:12-22,49`, all 5 `app/*/profile/page.tsx`, `actions/route-override.actions.ts:199-225`
2	**High**	Facilities role can create Ops/Facilities users, change roles, reset **any** user's password (incl. Super Admin), and hard-delete users	`actions/user-management.actions.ts:90, 498-545, 633-655, 816-848`
3	**High**	Password reset: token stored plaintext, URL built from `Host` header, TTL 1h (spec 30 min), other sessions not revoked	`actions/password-reset.actions.ts:16,44,49-52,90,103`

#	Severity	Issue	Where
4	**High**	No `AuditLog` model or `audit()` service at all (spec rule 7)	`prisma/schema.prisma`
5	**High**	No timeouts on OSRM/Nominatim `fetch`; one hung socket stalls RUN MAPPING	`lib/routing/osrm.ts:139-145`, `lib/routing/geocoder.ts:45`
6	**High**	Corridor sweep awaits OSRM legs serially in nested loops (≈n^2 serial calls on cold cache)	`lib/routing/algorithms.ts:451-475, 493-511`
7	**High**	`trainRoutePlannerFromHistory` loads **all** RoutePlans (any status, full includes) on every run; non-atomic `deleteMany` + `createMany`	`actions/route-learning.actions.ts:120-131, 206, 237-240`
8	**High**	Route Setting page performs an unbounded `Promise.all` of `vehicle.upsert` **on every GET**	`app/facilities/route-setting/page.tsx:71-89`
9	**High**	`corridor-core.ts` duplicates ~700 lines of `algorithms.ts` and is imported by nothing (drift risk)	`lib/routing/corridor-core.ts`
10	**Medium**	Routing timestamps depend on server timezone (`setHours` / `getHours`), not `Asia/Dhaka`	`lib/routing/algorithms.ts:127-132, 791, 841, 1082`, `actions/route-learning.actions.ts:50-53`

Mechanical checks run

Check	Result
`npx tsc --noEmit`	✅ 0 errors
`npx eslint .`	❌ **7 errors, 9 warnings** (details §5)
`any` / `as any`	✅ 0
`@ts-ignore` / `@ts-expect-error`	✅ 0
`eslint-disable`	7 (5 × `no-img-element`, 2 × `exhaustive-deps`)
`as unknown as`	11
Test runner / `npm test`	❌ none configured (19 `node:test` cases run only manually)

---

1. Architecture & Folder Structure

Current state

The top-level split is sound and idiomatic for the App Router: role-scoped route segments (`app/{employee,corporate-employee,operations,facilities,fixed-arrival,super-admin,(auth)}`), Server Actions in `actions/*.actions.ts`, role-scoped UI in `components/{role}/`, shared primitives in `components/ui` (shadcn) and `components/shared`, and a `lib/` with clear sub-domains (`auth`, `rbac`, `queries`, `routing`, `validation`, `constants`, `db`, `date`). No component imports `prisma`; the three client-side imports of `lib/` are type-only.

The weak points are **boundary discipline** and **duplication**:

- **Pages query the DB directly.** 28 of 39 pages import `prisma`; only 12 use `lib/queries/*`. Heaviest inline-query pages: `app/super-admin/dashboard/page.tsx` (7 queries), `app/facilities/route-setting/page.tsx` (6), `app/operations/dashboard/page.tsx` (5). The same `prisma.vehicleDriver.findMany` is repeated in 4 pages (`facilities/adhoc:24`, `route-planning:45`, `route-setting:30`, `vehicles:20`) — the new `lib/queries/fleet.ts` was written to replace these but is imported nowhere.
- **A page writes to the DB during render** — `app/facilities/route-setting/page.tsx:73` (`prisma.vehicle.upsert` per vehicle driver, every GET).
- **Server Actions used as read APIs.** Pages call `getWeekRouteOverview` (`app/facilities/automated-route-planner/page.tsx:4`) and `getVehicleShiftPins` (`app/facilities/route-setting/page.tsx:5`); client components fetch via `getGeneratedRouteDetails`, `getRoutePlanDetail`, `getRouteLearningStats` (`auto-route-planner-client.tsx` ~L168). These are uncached, serialized POSTs and each is a publicly reachable endpoint.
- **Client components bypass the server OSRM client** and hard-code the public demo server: `components/facilities/route-map.tsx:197`, `components/facilities/route-breakdown/index.tsx:110` (`https://router.project-osrm.org`), ignoring `env.OSRM_SERVER_URL`.
- **`server-only` guards are inconsistent.** 16 lib modules have it; `lib/prisma.ts`, `lib/auth/jwt.ts`, `lib/constants/notification-recipients.ts` and 6 DB-touching routing modules rely on transitive protection via `lib/env.ts`; **`lib/auth/password.ts` has no guard, direct or transitive.**
- **Copy-paste across role areas.** `employee` vs `corporate-employee` dashboard (22 differing lines), profile (12), adhoc (4); `operations` vs `facilities` allocation-history (4); two change-password forms (`components/employee/` vs `components/auth/`). All 6 role layouts repeat the same `NavSidebar + <main>` shell with hard-coded accent classes (`bg-[rgba(112,188,204,0.15)] text-[#70bccc]`) instead of design tokens.
- **Error/loading boundaries** exist only at the root (`app/loading.tsx`, `app/error.tsx`, `app/global-error.tsx`); no role segment has its own, so one failing widget blanks the whole shell.
- **Component modularity.** Oversized client files mixing fetching, mutation, DnD and rendering:

File	Lines	Notes
`components/facilities/auto-route-planner-client.tsx`	1,908	`GeneratedRouteCard` alone ≈1,035 lines / 18 `useState`; imports 4 action modules
`components/facilities/route-setting/route-setting-client.tsx`	992	21 `useState`, 9 inline sub-components
`components/facilities/route-map.tsx`	935	fetches OSRM itself
`components/facilities/route-breakdown/VehicleGroupTable.tsx`	576	
`components/facilities/route-breakdown/index.tsx`	470	fetches OSRM itself

Refresh callbacks (`onRefresh`, `onChanged`, `onRouteUpdated`, `onTimeChange`) are prop-drilled 3–4 levels (`route-breakdown/index.tsx:29-41` → `VehicleGroupTable.tsx:25-37` → `ShiftRow.tsx:10-25`). - **Minor:** `proxy.ts:8` lists `/register` as public but no such route exists; 162/213 files are CRLF and the rest LF (no `.gitattributes`).

Where and How to Improve

- **Enforce the read boundary.** Move every page-level `prisma.*` call into `lib/queries/<domain>.ts` (e.g. `dashboards.ts`, `profile.ts`, `adhoc.ts`). Wire `lib/queries/fleet.ts` into the 4 facilities pages. Then add an ESLint `no-restricted-imports` rule banning `@/lib/prisma` in `app/**/page.tsx`, `app/**/layout.tsx` and `components/**`.
- **Move read "actions" out of `actions/`.** `get*` functions consumed by Server Components → `lib/queries`. Those needed by client components → either pass from the server component as props or expose a typed route handler (`app/api/.../route.ts`) with GET caching. Keep `actions/` for mutations only.
- **Delete the render-time write** in `app/facilities/route-setting/page.tsx:71-89`; perform the VehicleDriver→Vehicle sync inside the vehicle-driver mutation actions (`actions/vehicle-driver.actions.ts`) within the same transaction.
- **Proxy browser OSRM calls** through a server route (`app/api/routing/route/route.ts`) that uses `lib/routing/osrm.ts` (cache + env URL + throttle), and call that from `route-map.tsx` / `route-breakdown/index.tsx`.
- **Add `import "server-only"`** as the first line of `lib/prisma.ts`, `lib/auth/{jwt,password,session}.ts`, `lib/mailer.ts`, `lib/constants/notification-recipients.ts`, and `lib/routing/{algorithms,osrm,pickup-schedule,route-analysis,route-pattern,geocoder}.ts`.
- **De-duplicate role areas.** Extract `components/shared/{EmployeeDashboard,ProfilePage,AdHocPage,AllocationHistoryPage}.tsx` taking `role` / `basePath` props; make the `employee` and `corporate-employee` pages 5-line wrappers. Keep a single change-password form.
- **One `RoleShell` layout** + a `nav-config.ts` map (`role → items, accent token`); replace hard-coded hex accents with Tailwind theme tokens from the design system (`--accent`, `--accent-2`).
- **Add per-segment `loading.tsx` (skeletons) and `error.tsx`** under each role directory.
- **Split the mega-components:** `auto-route-planner-client.tsx` → `generated-route-card/` folder with `useRouteCardState`, `useTripDrag`, `RouteCardHeader`, `TripTable`; `route-setting-client.tsx` → one file per inline sub-component. Replace refresh prop-drilling with a small `RoutePlannerContext` (or `useReducer` + context).
- Add `.gitattributes` (`* text=auto eol=lf`) and drop `/register` from `proxy.ts` public paths.

---

2. System Design & Routing Logic

Current state

Module map (as-is):

Module	Role	Pure?
`lib/routing/algorithms.ts` (1,596)	Orchestrator: loads DB, geocodes, builds matrices, plans corridors + 10 PM routes, times, persists	❌ fused I/O + logic
`lib/routing/corridor-core.ts` (722, **untracked**)	Sync corridor planner with injected `LegRoadIndex` + `isSameOffice`	✅ pure — **but unused**

Module	Role	Pure?
`lib/routing/osrm.ts` (1,011)	HTTP client, DB caches, plus dead pure helpers	❌
`lib/routing/side-selection.ts`	Viterbi side choice, exhaustive ≤7! + Or-opt	✅ clean
`lib/routing/schedule-math.ts`, `manual-plan-training.ts`, `utils.ts`	Math / helpers	✅ (except dead `calculateCompletionTime` uses `new Date()`)
`lib/routing/pickup-schedule.ts`	Pure `computePickupSchedule` / `driveLegMinutes` + Prisma loaders	⚠️ mixed
`lib/routing/route-pattern.ts`	Pure pattern helpers (L44-145) + Prisma writers	⚠️ mixed
`lib/routing/route-analysis.ts`	I/O orchestration over `side-selection`	❌ (expected)
`lib/routing/geocoder.ts`	Nominatim fetch	❌ (expected)

`corridor-core.ts` vs `algorithms.ts`: `algorithms.ts` is byte-identical to HEAD, and `grep` finds **zero importers** of `corridor-core`. Its header (L17-20) claims "algorithms.ts imports from here" — currently false. The two files duplicate constants (cc 55-98 / alg 66-103), all shape types, `matrixDurationMin` / `matrixDistanceKm`, `distancePointToSegment/PolylineKm`, `arrivalMinutesFromLabel` / `dateForArrivalMinutes` / `hhmmToMinutes`, `poolForOffice`, `errorMessage` and `planCorridorRoutes` itself. The new copy is the better design (synchronous, injected data, `legRoadKm → number | null` with straight-line fallback), but until it is wired in, every tuning change risks silent divergence.

Purity leaks / misplaced decision logic:

- Pure functions trapped in I/O modules (importing them pulls in `@/lib/prisma`): `pickup-schedule.ts:135,152`, `route-pattern.ts:44-145`, `osrm.ts:961,985`.
- Heavy decision logic in action files: `actions/route-override.actions.ts:601-907` (`resolvePresetSequence`, `presetInsertIndex`, `resolveCanonicalPointId`, 176-line `buildRetimeUpdates`, `legDriveMinutes` — which duplicates `pickup-schedule.ts:135` and `algorithms.ts:1342-1358`); `actions/route-analysis.actions.ts:238-325` (pickup/departure math using **OSRM** leg minutes — contradicts the "trained legs only" policy used everywhere else); `actions/route-learning.actions.ts:133-235` (leg aggregation).
- `actions/route-override.actions.ts:19` imports `errorMessage` / `DYNAMIC_GROUP_CAPACITY` from `algorithms.ts`, dragging the whole Prisma-laden orchestrator into that action.
- **Helper sprawl:** `hhmmToMinutes` × 4 with different invalid-input semantics (0 vs null), arrival-label→minutes map × 4 (despite `arrivalMinutesForLabel` in `lib/constants/route-shifts`), `shiftKeyForLabel` × 3, bearing math × 2 (`osrm.ts:114` vs `utils.bearingDegrees`).
- **Timezone-dependent decisions:** `dateForArrivalMinutes` (`algorithms.ts:127-132`, `corridor-core.ts:330`), `departureTime10.setHours(20…)` (`algorithms.ts:841`), `setDate(getDate()+n)` (`algorithms.ts:791,1082`), read back with `getHours()` (`route-learning.actions.ts:50-53`, `route-analysis.actions.ts:320-323`). No `Asia/Dhaka` handling anywhere in routing — on a UTC host "23:00" is stored as 05:00 Dhaka.
- **Determinism:** `prisma.vehicle.findMany` (`algorithms.ts:1051`) and `weeklyScheduleDay.findMany` (L1022) lack `orderBy`; `nextVehicle()` and stable-sort ties therefore depend on DB return order.

- **Behaviour/doc mismatch:** docstrings (`algorithms.ts:354-356`, `corridor-core.ts:31-33`) say a candidate failing radius/detour is "SKIPPED", but code uses `break` (`algorithms.ts:597,610`; `corridor-core.ts:699,712`), ending growth of that route.
- **Learning feedback loop:** `trainRoutePlannerFromHistory` (`route-learning.actions.ts:121-131`) has no status filter and runs *before* drafts are cleared (`route-generation.actions.ts:200` vs `218`), so the planner learns from its own DRAFT/cancelled output; the hard-coded 20:00 placeholder departure (`algorithms.ts:840-841`) is learned as a real office→first-stop leg (`route-learning.actions.ts:179-183`).
- **God functions (>~150 lines):** `generateCorridorRoutes` (`algorithms.ts:1007-1512`, ~505), `generate10PMRoutes` (648-931, ~283), `planCorridorRoutes` (375-620, ~245), `moveTripToRoute` (`route-override.actions.ts:1063-1293`, ~230), `buildRetimeUpdates` (705-881), `trainRoutePlannerFromHistory` (117-269), `reorderRouteTrips` (914-1062), `finalizeRoute` (324-470).
- **Magic numbers scattered:** 25 km/h fallback speed (5 sites), 0.05 km zigzag floor, 240-min leg cap vs `MAX_LEG_MINUTES`, fallback office `"Regnum"` (5 sites), 20:00 departure, 23:00 default arrival, 5-min unknown-leg buffer, 500 ms × 4 retry, tx timeouts 30 000/90 000, 135° loop threshold.

Strengths: thorough rationale comments; `side-selection.ts` is a model pure module ($O(n \cdot k^2)$ Viterbi, tie-stable); per-office try/catch isolation; OSRM outage degrades to haversine; batched cache reads/writes; in-flight polyline de-dupe; corridor path persists with `createMany` and compensating rollback.

Where and How to Improve

- **Finish the corridor-core extraction (highest leverage):** 1. In `generateCorridorRoutes`, gather every candidate leg, prefetch them into a `LegRoadIndex` with bounded concurrency (e.g. `p-limit(4)`; failures → `null`). 2. Call the synchronous `planCorridorRoutes` from `corridor-core.ts` with `isSameOffice = (o, k) => normalizeOfficeLocation(o) === k`. 3. Add golden tests comparing old vs new planner output on recorded fixtures, then delete the duplicated block in `algorithms.ts` (L66-139, 163-341, 375-620, 629, 935). 4. Re-point `route-override.actions.ts:19` imports to `corridor-core`.
- **Adopt a strict "core / shell" layout** for `lib/routing/`:
- `*-core.ts` = pure (no `prisma`, `fetch`, `env`, `Date.now`, `Math.random`): `corridor-core.ts`, `pickup-schedule-core.ts` (move L135-260), `route-pattern-core.ts` (L44-145), `retime-core.ts` (from `route-override.actions.ts:601-907`), `leg-aggregation.ts` (from `route-learning.actions.ts:133-235`, next to `manual-plan-training.ts`), `timing-core.ts` (from `algorithms.ts:1324-1416` → `timeCorridorRoute()`).
- shells = `algorithms.ts` (orchestration only), `osrm.ts`, `geocoder.ts`, `*-repo.ts` loaders/writers.
- Enforce with ESLint `no-restricted-imports` on `lib/routing/*-core.ts` (ban `@/lib/prisma`, `@/lib/env`, `@/lib/logger`, `./osrm`, `./geocoder`).
- **Create** `lib/routing/constants.ts` for every magic number above and `lib/routing/time.ts` with a single `hhmmToMinutes` / `arrivalMinutes` / `shiftKeyForLabel`; delete the 3–4 copies.
- **Make time zone explicit:** add `lib/date/zoned.ts` with `zonedMinutesToUtc(dayDate, minutes, "Asia/Dhaka")` and `utcToZonedMinutes(...)`; replace every `setHours` / `getHours` / `setDate` in routing and route actions.
- **Make planning deterministic:** add `orderBy: { vehicleNumber: "asc" }` / `{ id: "asc" }` at `algorithms.ts:1022,1051` and explicit tie-breakers (employee id) in every sort comparator.
- **Fix the learning loop:** filter `status: { in: ["FINALIZED","EXECUTED"] }`, exclude the 20:00 placeholder departure from leg learning, wrap `deleteMany` + `createMany` in `$transaction`, and train incrementally (cursor on `updatedAt`) instead of full rebuild.

- **Decide the OSRM-vs-trained-legs policy** for `route-analysis.actions.ts:238-325` and make it consistent with the rest of the pipeline.
- **Resolve `break` vs "skip" semantics** at `algorithms.ts:597,610` — either change to `continue` or correct the docstrings, and add a test that pins the chosen behaviour.
- **Break up the god functions:** `generateCorridorRoutes` → `loadInputs → resolveHomes → buildMatrix → buildLegIndex → plan → time → persist`, each independently testable.

---

3. Performance & Scalability

Current state

I/O concurrency

- **Serial OSRM in nested loops (biggest latency hot spot):** the corridor sweep awaits `legRoadKm` per route × member × position (`algorithms.ts:451-475, 493-511`). With the 350 ms public-server throttle, ~60 homes ≈ 3,500 legs ≈ ~20 min cold.
- **No fetch timeouts:** `osrm.ts:139-145` and `geocoder.ts:45` — no `AbortSignal`. Retries (L147-149, 177-180) are linear 500·n ms without jitter, and network `TypeError`s are not retried.
- **Racy throttle:** `lastRequestAt` check-then-set across `await sleep` (`osrm.ts:107-109, 136-142`) lets concurrent callers fire together; the throttle is disabled whenever `OSRM_SERVER_URL` is set — and `.env.example:39` sets it to the public demo URL. The promised "bounded concurrency" limiter (L102-105) does not exist.
- **Distance matrix:** `getDistanceMatrix` (`osrm.ts:335-342`) sends `[...origins, ...destinations]` and the caller passes the same list twice (`algorithms.ts:1157`) → 2n coordinates; >~49 riders exceeds the public table cap and silently falls back to haversine (`algorithms.ts:1158-1185`) with no surfaced warning. Expired `DistanceMatrixCache` rows are never refreshed because `createMany({ skipDuplicates })` (`osrm.ts:296-299`) is blocked by the expired row.
- **Independent sequential awaits** that should be `Promise.all`: `app/operations/weekly-requests/page.tsx:40,54,73,86`; `app/facilities/route-planning/page.tsx:45,51,66,112`; `app/operations/adhoc/page.tsx:10,23`; `app/operations/dashboard/page.tsx:27,46`; `app/employee/profile/page.tsx:14,18`; `actions/route-generation.actions.ts:773,788`; `actions/route-override.actions.ts:1074/1093, 1193/1201, 1256/1261`; `lib/routing/route-analysis.ts:341/346, 391/423`; `actions/manual-route-plan.actions.ts:240,248`. Different days in `route-generation.actions.ts:231-263` run serially although only 11 PM→10 PM within a day is dependent.
- **Unbounded `Promise.all` (Neon pool pressure):** `ensureFixedArrivalSchedules` ≈6×N upserts (`route-generation.actions.ts:92-133`); `facilities-profile.actions.ts:54-71`; `app/facilities/route-setting/page.tsx:71`; bulk-user bcrypt over all rows (`user-management.actions.ts:323-333`, pure-JS `bcryptjs` → blocks the event loop ~80 ms/row and hashes rows later skipped).
- **`$transaction` usage is healthy:** all are batch/array form with no OSRM/email inside. The global `timeout: 90000` in `lib/prisma.ts` can hold a pooled connection for 90 s.

N+1 / over-fetch

- 10 PM path: up to 2 `pickupPoint.findFirst` per employee (`algorithms.ts:704-731`); per vehicle `findMany` (L760) + `findUnique` (L774) + `ensurePickupPointSchedule` (L779, re-reads vehicle and calls

`loadLearnedLegs`); per-trip `tripAssignment.create` loop (L895-910), non-transactional delete-then-create (L868-892) with `.catch(() => undefined)`.

- `loadLearnedLegs` (`pickup-schedule.ts:293-308`) scans the full `routeLegLearning` table + all users with addresses, re-run per vehicle × shift × day.
- Per corridor route: `routeTemplate.findUnique` (L1293) + `routePlan.findUnique` (L1423).
- Day-sheet bulk save: ~4 sequential round trips per row + one `after()` SMTP send per row (`route-assignment.actions.ts:60-77, 88-93, 300-330`); the `vehicleDriver.findUnique` at L93 is redundant with the map at L276-282.
- Bulk weekly upload: `create` + 5 upserts per row (`weekly-schedule.actions.ts:668-718`).
- `trainRoutePlannerFromHistory` (`route-learning.actions.ts:120-131, 206`): every RoutePlan with trips, full employees, addresses and the large `routeAnalysis` JSON, on every RUN MAPPING.
- **Unbounded reads:** `app/operations/adhoc/page.tsx:10` (no `take`), `app/facilities/adhoc/page.tsx:12` (includes ever-growing REJECTED), `app/facilities/profile-approvals/page.tsx:12`; `getAllUsers` / `getEmployeeDirectory` (`lib/queries/users.ts:21,89`) filter `q` in memory.
- **Correctness bug:** `lib/queries/reports.ts:35-56` applies the date range **in memory after** `take: 1000`, so older ranges silently return nothing.
- `distinct: ["weekStartDate"]` (`weekly-requests/page.tsx:40`, `route-planning/page.tsx:51`) reads all WeeklySchedule rows; `getRouteDayStats` includes trips only to count them; `getGeneratedRouteDetails` ships the whole week's `routeAnalysis` geometry JSON on every refresh.

Caching (`unstable_cache` / tags)

- `lib/cache-tags.ts:1-20` documents `updateTag()` invalidation, but there are **zero** `updateTag` / `revalidateTag` calls in `actions/`, `lib/` or `app/`. The only `unstable_cache` readers (`lib/queries/fleet.ts:27,77`) are imported nowhere. If wired as-is, data would be stale for up to 300 s after every mutation.
- `getTransportSettings` (`lib/queries/settings.ts:19`) is listed as cached but is not — a singleton row read on 5 pages + 2 actions.
- No per-user data is cached under a shared key (good); `revalidatePath('/', 'layout')` is never used (good). All invalidation is currently via `revalidatePath`.
- In-process caches `polylineCache` (`osrm.ts:561`) and geocoder `cache` (`geocoder.ts:21`) are unbounded Maps; the DB polyline key ignores bearing while the memory key includes it (`osrm.ts:605-623` vs 761); geocoder cache key ignores `limit`.

Indexes (`schema.prisma` + new migration)

- **Redundant:** `TripAssignment @@index([routePlanId])` and `[routePlanId, vehicleId]` (L584-585) — covered by `@@unique([routePlanId, employeeId, weeklyScheduleDayId])`; `PickupPoint @@index([vehicleId, sequence])` (L410) duplicates its `@@unique` (L409); `[isActive]` (L415) is a low-selectivity boolean; single-column `AdHocRequest[status]` (L261), `EmailNotificationLog[routeAssignmentId]` (L323), `WeeklySchedule[status]` (L207), `User[role]` (L71) are now prefixes of new composites; `RoutePlan[vehicleId, dayDate, arrivalTime]` (L528) duplicates the unique at L522.
- **Missing FK indexes** (Postgres does not auto-index FKs): `RoutePlan.previousRoutePlanId` (L499 — checked on every draft delete), `RoutePlanOverride.routePlanId` (L591 — cascade target),

`TripAssignment.pickupPointId` (L549) / `pickupAddressId` (L552), `ManualRoutePlanEntry.employeeId` (L688, queried with `IN`), `reviewedById` on the three change-request tables.

- **Un-indexable lookups:** `mode: "insensitive"` on `pickupPoint.name` (`algorithms.ts:721-731`, `facilities-profile.actions.ts:142-148`) and on email/fullName `IN` (`manual-route-plan.actions.ts:240-251`).

Email

- Most sends correctly use `after()` (route assignment, finalize, bulk uploads, on-behalf, password reset, profile).
- `actions/weekly-schedule.actions.ts:295` calls `sendScheduleSubmittedNotice(...)` with neither `await` nor `after()` — can be dropped on serverless.
- Bulk paths use unbounded `Promise.allSettled` over recipients with a non-pooled transporter (`lib/mailer.ts:16-26`) → N simultaneous TLS connections.
- `finalizeRoute` stamps `emailSentAt` even when sends fail, because `sendMailBackground` never rejects, and writes no `EmailNotificationLog` (`route-override.actions.ts:440-449`).
- `lib/auth/session.ts:53` `void deleteExpiredSessions()` should be in `after()`.

Client bundle: fine — Leaflet is `next/dynamic({ ssr: false })` in both consumers; layouts are server components; only concern is the 1,908-line single client tree.

Where and How to Improve

- **Add timeouts + a real limiter to all outbound HTTP:** `fetch(url, { signal: AbortSignal.timeout(10_000) })` in `osrm.ts` and `geocoder.ts`; wrap in try/catch to retry network errors with exponential backoff + jitter; replace the `lastRequestAt` throttle with a promise-chain mutex / token bucket keyed by **hostname** (throttle `router.project-osrm.org` and `nominatim.openstreetmap.org` regardless of env), plus `p-limit(n)` for fan-out.
- **Prefetch legs in parallel, plan synchronously** (see §2) — turns ~n^2 serial OSRM awaits into bounded-parallel batches + pure CPU.
- **Fix the matrix call:** dedupe coordinates and use OSRM `sources=` / `destinations=` indices on a single coordinate list; chunk above the server cap; push a user-visible warning into `errors` when falling back to haversine; on expired cache rows, `deleteMany` expired pairs before `createMany` (or upsert); add a cleanup job for expired cache rows.
- **Kill the N+1s in generation:** preload active pickup points, vehicles, route templates and existing RoutePlans once per day/shift into Maps; memoize `loadLearnedLegs` per run per `shiftKey`; replace the per-trip `create` loop with `createMany` inside `$transaction` (matching the corridor path).
- **Bound fan-outs:** `p-limit(5–10)` for `ensureFixedArrivalSchedules` (or nested `create` + `createMany`), `facilities-profile.actions.ts:54-71`, and bulk bcrypt (hash only rows that will be created; consider native `bcrypt`/argon2).
- **Batch the day-sheet save:** one `findMany` for all days, one `$transaction([...upserts])`, one `createMany` for email logs, and a single `after()` that sends all emails through a pooled transporter.
- **Parallelize independent awaits** at every location listed above with `Promise.all`, and run different days in `route-generation.actions.ts:231-263` with limited concurrency.
- **Bound every list query:** add `take` + cursor pagination to ad-hoc / profile-approvals pages, exclude old REJECTED rows by date, push `q` filtering into Prisma `where` (`contains`, `mode: "insensitive"` + a

trigram/`lower()` index), and **move the date range into `where` in `lib/queries/reports.ts`** (as `allocation-history.ts:58-66` already does).

- **Slim `trainRoutePlannerFromHistory`:** `select` only `arrivalTime`, `officeLocation`, `departureTime`, trips' `sequenceInTrip`/`pickupTime`/stop keys; window to the last N weeks; filter by status; wrap rebuild in `$transaction`.
- **Wire the cache layer correctly before using it:**
- `lib/queries/fleet.ts` → used by the 4 facilities pages and `route-assignment.actions.ts:276`.
- Cache `getTransportSettings` under tag `settings`.
- In mutation Server Actions call `updateTag(CACHE_TAGS.x)` (read-your-writes): `vehicle-driver.actions.ts:60,94,190,211`; `pickup-point.actions.ts` (L33,117,188,220); `route-setting.actions.ts` (L77-107,127,208,331,434,480); `facilities-profile.actions.ts` (L54-71,103-110,150-157); `route-analysis.actions.ts:379`; `settings.actions.ts:47`.
- Outside Server Actions (e.g. `pickup-schedule.ts:385,539` running in `after()`), use `revalidateTag(CACHE_TAGS.pickupPoints, "max")` (Next 16 two-arg form).
- Add an LRU cap/TTL to `polylineCache` and the geocoder cache; include bearing in the DB polyline key and `limit` in the geocoder key.
- **Index migration (follow-up):**
- **Add:** `RoutePlan(previousRoutePlanId)`, `RoutePlanOverride(routePlanId)`, `TripAssignment(pickupPointId)`, `TripAssignment(pickupAddressId)`, `ManualRoutePlanEntry(employeeId)`, `reviewedById` on change-request tables; `lower()` expression indexes (raw SQL in migration) for case-insensitive name/email lookups, or store normalized columns.
- **Drop (after confirming with `pg_stat_user_indexes`):** `TripAssignment[routePlanId]`, `[routePlanId, vehicleId]`, `PickupPoint[vehicleId, sequence]`, `PickupPoint[isActive]`, `RoutePlan[vehicleId, dayDate, arrivalTime]` (replace with plain `[vehicleId]`), and the single-column prefixes superseded by composites.
- **Email reliability:** wrap `weekly-schedule.actions.ts:295` in `after()`; enable `pool: true, maxConnections: 3, connectionTimeout, socketTimeout` in `lib/mailer.ts`; bound bulk sends; in `finalizeRoute` use `trySendMail` + `EmailNotificationLog` and stamp `emailSentAt` only on success; move `deleteExpiredSessions` into `after()`.
- **Move RUN MAPPING off the request path** (long-term): enqueue a job (e.g. Vercel Queues / Inngest / a `RouteGenerationJob` table + cron worker) and poll status from the UI.
- Lower the global Prisma transaction `timeout` from 90 s to the minimum each call site needs (pass per-call `timeout` where larger is genuinely required).

---

4. Security & Best Practices

Current state

What is done well (keep it):

- **Guard-first RBAC is consistent:** all 73 non-public Server Actions call `requireRole`/`requireAuth` as the first statement, **outside** `try/catch`, before any I/O. The 4 public entry points (login, logout, request-reset, reset) are intentional.

- **Error sanitization is consistent:** every catch uses `lib/actions/action-error.ts` (logs raw, returns generic, rethrows `NEXT_REDIRECT`, maps P2002 to a friendly message). No `error.message` or Prisma error text is returned from any action.
- **No IDOR:** every employee-scoped action uses `session.id`, never a client-supplied id (`submitAdHocRequest`, `updateProfile`, `saveFixedArrivalTime`, `submitCorporateWeeklySchedule`, `submitMySchedule`, `changePasswordAction`, `selfChangePasswordAction`).
- **No mass assignment:** all Prisma `data` objects are built from whitelisted fields.
- **No unsafe SQL:** the only raw query is the tagged-template `$queryRaw`SELECT 1`` (`app/api/health/route.ts:24`).
- **Sessions:** HS256 via jose, secret from env (no fallback), `jti` validated against a DB `Session` row on every `getSession` (revocation + `isActive` honoured), cookie `httpOnly` + `secure` (prod) + `sameSite=lax`. Security headers set in `next.config.ts` (nosniff, frame DENY, Referrer-Policy, HSTS in prod, `poweredByHeader: false`). No open redirects. No secrets in tracked code. Forgot-password response is enumeration-safe.

Per-file guard coverage:

File	Exports	Guard-first	Allowed roles	zod	Audit
adhoc	4	4/4	EMP+CORP / OPS	✗	✗
auth	4	2 public + 2/2	—	✓	✗
facilities-profile	4	4/4	FAC (3), FAC+SA (1)	✗	✗
fixed-arrival	1	1/1	FIXED_ARRIVAL	✗	✗
manual-route-plan	1	1/1	FAC+SA	✗	✗
on-behalf	2	2/2	OPS	✗	✗
password-reset	2	public	—	✓	✗
pickup-point-change	2	2/2	FAC	✗	✗
pickup-point	4	4/4	FAC+SA	✗	✗
profile	1	1/1	EMP+CORP	✗	✗
route-analysis	3	3/3	FAC+SA	✗	2/3 (override log)
route-assignment	5	5/5	FAC (4), FAC+SA (1)	✗	✗
route-generation	7	7/7	FAC+SA	✗	✗
route-learning	3	3/3	FAC+SA	n/a	✗
route-override	9	9/9	FAC+SA	✗	5/9 (override log)
route-setting	7	7/7	FAC+SA	partial/manual	✗
settings	1	1/1	SA	manual	✗
user-management	7	7/7	**FAC allowed on create/bulk/resend/setPw/delete**	1/7	✗

File	Exports	Guard-first	Allowed roles	zod	Audit
vehicle-driver	4	4/4	FAC	manual	✗
weekly-schedule	6	6/6	CORP / OPS	manual	✗

Findings:

ID	Sev	Finding	Location
S1	**Critical**	**Password hashes and live reset tokens reach the browser.** Full `User` rows (`include: { employee: true }` / `findUniqueOrThrow` with no `select`) are passed as props to `"use client"` components and serialized into the RSC payload: Operations sees *every* requester's hash on the ad-hoc page; Facilities likewise; each profile page ships the user's own hash/token. `adjustPickupTime` returns `assignment: updated` with `employee: true` to two client callers. Combined with S3 (plaintext tokens), any outstanding reset token = account takeover.	`app/operations/adhoc/page.tsx:10-16,46` ; `app/facilities/adhoc/page.tsx:12-22,49` ; `app/{employee,corporate-employee,operations,facilities,fixed-arrival}/profile/page.tsx` → `components/shared/profile-hero-card.tsx` ; `actions/route-override.actions.ts:199-225` (callers `auto-route-planner-client.tsx:1136` , `route-breakdown/hooks/useDragActions.ts:103`); also uncalled exports `reassignEmployeeToVehicle` (L102-117), `getRouteDetails` (L512-538), `getPendingRoutes` (`route-generation.actions.ts:484-497`)
S2	**High**	**Privilege escalation / account takeover by Facilities.** `createUserAction` allows FACILITIES and only blocks SUPER_ADMIN targets (L90, 112) → Facilities can mint OPERATIONS/FACILITIES accounts with chosen passwords. `bulkCreateUsersAction` update branch can change roles (L520) and overwrite passwords (L536) of any non-SA user. `resendCredentialsAction` (L633-655) has no target-role check → Facilities can reset a **Super Admin's** password. `deleteUserAction` (L816-848) lets Facilities hard-delete Ops/Facilities users and their history. (`setUserPasswordAction:775` correctly limits FAC to EMPLOYEE targets — apply the same rule everywhere.) Spec: only Super Admin creates users.	`actions/user-management.actions.ts`
S3	**High**	**Password-reset hardening.** Token stored and compared in plaintext (L44, L90, `app/(auth)/reset-password/[token]/page.tsx:12`); reset URL built from the request `Host` header → host-header poisoning (L49-52); TTL 1 h vs spec 30 min (L16); find-then-update is non-atomic.	`actions/password-reset.actions.ts`

ID	Sev	Finding	Location
S4	**High**	**No audit log.** No `AuditLog` model; only partial trails via `RoutePlanOverride` and `reviewedById/At`. User create/delete/role/password/activation, settings, profile edits, approvals, finalize/cancel and vehicle CRUD are unaudited — and deletes are hard deletes.	`prisma/schema.prisma`
S5	Medium	**Credential change doesn't revoke other sessions.** `rotateSession` destroys only the current session; `resetPasswordAction` (L103), `setUserPasswordAction` (`user-management:780`), `resendCredentialsAction` (L652), bulk overwrite (L536) and `updateUserRoleAction` never delete the target's `Session` rows.	`lib/auth/session.ts:139-142` + callers
S6	Medium	**No rate limiting / lockout** on `loginAction`, `requestPasswordResetAction` (email bombing), `resetPasswordAction`.	`actions/auth.actions.ts`, `actions/password-reset.actions.ts`
S7	Medium	**Admin-set passwords emailed in plaintext and `mustChangePassword: false`.**	`actions/user-management.actions.ts:150,155-161,568,593-598`
S8	Medium	**HTML injection in emails** — user-controlled name/address/reason interpolated unescaped into HTML sent to staff inboxes.	`actions/profile.actions.ts:114,118,122`; `on-behalf.actions.ts:251`; `weekly-schedule.actions.ts:65`; `route-override.actions.ts:408,421`; `lib/email-templates.ts:46-63`
S9	Medium	**Runtime validation is thin.** zod used only in auth/password-reset + `setUserPasswordSchema`. Object-arg actions trust TS types: `pickup-point.actions.ts:171-177` (`baseETA`, `name`), `route-analysis.actions.ts:89-180` (unbounded `geometry` / `viaPoints` / `totals` JSON persisted), route-override args, `facilities-profile:42-44`; no `max()` on free text.	`actions/*`
S10	Medium	**Business-rule gaps with security impact.** Employees change their own address directly (`profile.actions.ts:83-96`) — spec requires Operations approval. `upsertAssignmentForWeeklyDay` doesn't check APPROVED / `needsTransport` (`route-assignment.actions.ts:89-114`); `replicateAcrossWeek` overwrites already-allocated days (L365-376).	as listed
S11	Medium	**Upload DoS.** No explicit file-size / row caps on CSV uploads (only Next's default 1 MB body); parallel bcrypt of every row. Dev "test mode" skips email/phone validation outside prod (`user-management.actions.ts:63-70`, mirrored in `components/super-admin/add-employee-dialog.tsx:30`) — TODO says remove.	`actions/user-management.actions.ts:323-333`
S12	Low	Unauthenticated `/api/health` returns raw DB error text and config warnings (e.g. "SESSION_SECRET is not set").	`app/api/health/route.ts:27,39`
S13	Low	Login timing oracle — returns before bcrypt when the user is missing.	`actions/auth.actions.ts:41`

ID	Sev	Finding	Location
S14	Low	Weak policies: `SESSION_SECRET` min 16 chars (`lib/env.ts:29`); password rule only `min(8)` (spec: 1 uppercase + 1 number).	`lib/env.ts` , `lib/validation/schemas.ts`
S15	Low	`mustChangePassword` enforced only in `proxy.ts:306` , not in `requireRole` ; Server Actions can be POSTed from any path.	`lib/rbac/guards.ts`
S16	Low	Raw SMTP error strings returned to staff.	`user-management.actions.ts:670` , `route-assignment.actions.ts:517`
S17	Low	Check-then-update races: approve/reject ad-hoc and pickup-point-change don't put `status: PENDING` in the `where` ; `approvePickupPointChange` (L44-56) does 2 writes without a transaction; `rejectSchedule` (`weekly-schedule:437-452`) never checks status (APPROVED can be rejected).	as listed
S18	Low	Role lists drift from spec/comments: approvals and on-behalf are `[OPERATIONS]` only (spec includes SA); several Facilities actions exclude SA despite the `proxy.ts:254` comment claiming "all 41". Role change leaves the JWT role claim stale (DB guards remain correct).	`actions/*` , `proxy.ts:254`
S19	Low	Intentional user-facing messages thrown inside `try` are replaced by the generic sanitized message (safe, but poor UX).	`route-override.actions.ts:153,161,259,483` ; `route-generation.actions.ts:276,344`
S20	Low	JWT payload cast with `as unknown as` rather than schema-validated; no refresh / sliding session (spec: 30 min access + 7 d refresh).	`lib/auth/jwt.ts:42`

Where and How to Improve

- **S1 — never send Prisma rows to the client.**
- Replace every `employee: true` / `user: true` / bare `findUniqueOrThrow` that flows to a client component or action return with an explicit `select` (e.g. `{ id, fullName, employeeCode, email, role, contactNumber }`).
- Introduce DTO mappers in `lib/queries/*` (`toEmployeeSummary(user)`) and return only DTOs from Server Actions.
- Add a Prisma client extension (`$extends({ result: { user: { passwordHash: { needs: {}, compute: () => undefined } } } })`) or `omit: { user: { passwordHash: true, passwordResetToken: true, passwordResetExpires: true } }` as a global default (Prisma 7 `omit` client option) so the hash can never be loaded accidentally.
- Also slim `lib/auth/session.ts` `include: { user: true }` (runs on every guarded request).
- Delete the uncalled exported actions (`getPendingRoutes` , `getRouteDayStats` , `reassignEmployeeToVehicle` , `swapEmployees` , `cancelRoute` , `getRouteDetails` , `setPickupPointActive` , `createVehicle`) — each is a reachable POST endpoint.

- **S2 — lock down user management.** Make `createUserAction`, `bulkCreateUsersAction`, `deleteUserAction`, `resendCredentialsAction` `[SUPER_ADMIN]` only. If Facilities genuinely needs onboarding, add a shared `assertCanManageTarget(actor, targetRole, newRole?)` helper that restricts Facilities to `EMPLOYEE` / `FIXED_ARRIVAL` for **both** the new role and the existing target on every path (create, bulk-update branch, resend, set-password, delete).
- **S3 — reset flow.** Store `sha256(token)` only; compare hashes; set TTL to 30 min; consume atomically with `updateMany({ where: { passwordResetTokenHash, passwordResetExpires: { gt: now } }, data: { … token: null } })` and check `count === 1`; build URLs from `env.APP_URL` / `FRONTEND_URL`, never `headers().get("host")`.
- **S4 — audit log.** Add `AuditLog { id, actorId, actorRole, action, entityType, entityId, beforeJson?, afterJson?, description, createdAt }` with `@@index([createdAt])`, `@@index([actorId])`, `@@index([entityType, entityId])`; create `lib/audit.ts` exporting `audit(tx, {...})` and call it inside the same `$transaction` as every mutation. Prefer soft-delete (`deletedAt`) for users.
- **S5 —** on every credential or role change: `prisma.session.deleteMany({ where: { userId } })` (keep current session only for self-change).
- **S6 —** add per-IP + per-email rate limits (Upstash Ratelimit, or a `LoginAttempt` table with sliding window) and temporary lockout after N failures.
- **S7 —** set `mustChangePassword: true` for admin-created accounts, or (better) email a one-time set-password link instead of a plaintext password.
- **S8 —** add `lib/html.ts` `escapeHtml()` and apply it to every interpolated value in email templates (or move templates to React Email, which escapes by default).
- **S9 —** one zod schema per action in `lib/validation/schemas.ts` (with `.max()` on strings/arrays and a size cap on JSON geometry); parse at the top of each action right after the guard. A small `withAction(roles, schema, handler)` wrapper would standardize guard → parse → try/sanitize.
- **S10 —** route address changes through an `AddressChangeRequest` approval; in `upsertAssignmentForWeeklyDay` require `schedule.status === APPROVED && day.needsTransport`; make `replicateAcrossWeek` skip already-allocated days.
- **S11 —** check `file.size` (e.g. ≤ 2 MB) and row count (e.g. ≤ 2,000); validate rows first, then hash with bounded concurrency; remove the `isProd()` test-mode bypass (or gate it behind an explicit `ALLOW_TEST_ACCOUNTS=1` that `env.ts` refuses in production).
- **S12–S20 —** `/api/health` returns only `{ status, db: "ok"|"down" }` publicly; compare against a dummy bcrypt hash when the user is missing; raise `SESSION_SECRET` min to 32 and enforce the spec password regex; enforce `mustChangePassword` inside `requireRole` for mutations; map SMTP errors to generic text; use `updateMany({ where: { id, status: PENDING } })` + count check inside `$transaction` for all approvals; reconcile role lists with the spec and fix the `proxy.ts:254` comment; return `{ error }` for intentional user messages instead of throwing; validate the JWT payload with zod.

---

5. Code Quality & Technical Debt

Current state

TypeScript strictness: `strict: true` and **zero** `any` / `as any` / `@ts-ignore` / `@ts-expect-error` — excellent. Remaining escape hatches: 11 × `as unknown as` (`actions/route-analysis.actions.ts:68,159`; `route-`

`breakdown/DragOverlay.tsx:100`; `lib/auth/jwt.ts:42`; `lib/prisma.ts:9,57`; `lib/routing/algorithms.ts:679,985,1034`; `lib/routing/corridor-core.ts:428`; `lib/routing/osrm.ts:328` — `null as unknown as DistanceElement` hides a nullable), 8 × non-null `!` (`components/employee/weekly-schedule-table.tsx:71×2,127,225`; `route-breakdown/index.tsx:442`; `lib/routing/pickup-schedule.ts:174,175,483,484`), ~27 plain `as Type` casts. Missing compiler flags: `noUncheckedIndexedAccess`, `noUnusedLocals`, `noUnusedParameters`, `noImplicitReturns`, `noFallthroughCasesInSwitch`, `noImplicitOverride`. Target is `ES2017` with `allowJs: true`.

ESLint (`npx eslint .`): 7 errors, 9 warnings.

File:line	Rule	Issue
`app/(auth)/reset-password/[token]/page.tsx:17`	react-compiler purity	`Date.now()` during render
`components/corporate-employee/weekly-schedule-form.tsx:57`	set-state-in-effect	sync `setState` in effect
`components/facilities/pickup-point-approval-card.tsx:65`	set-state-in-effect	″
`components/facilities/vehicle-driver-form-dialog.tsx:51`	set-state-in-effect	″
`components/operations/my-schedule-form.tsx:57`	set-state-in-effect	″
`components/operations/reject-reason-dialog.tsx:43`	set-state-in-effect	″
`components/shared/set-password-dialog.tsx:43`	set-state-in-effect	″
`components/facilities/auto-route-planner-client.tsx:874`	no-unused-vars (warn)	`GeneratedRouteCard` flagged unused
`components/facilities/route-breakdown/DragOverlay.tsx:33,49,54`	unused / exhaustive-deps (warn)	
`components/facilities/route-breakdown/EmptyVehicleDropZone.tsx:19,21`	unused (warn)	`onDrop`, `setIsHovered`
`components/facilities/route-breakdown/index.tsx:46`	unused (warn)	`onRouteSelect`
`hooks/use-toast.ts:13`	unused (warn)	`id`
`scripts/maintenance/repair-sync-point-times.ts:26`	unused (warn)	`DAY_NAMES`

`eslint-disable` × 7: 5 × `@next/next/no-img-element` (`app/(auth)/layout.tsx:8,17`, `components/shared/mobile-nav.tsx:61`, `components/shared/nav-sidebar.tsx:223,314`), 2 × `react-hooks/exhaustive-deps` (`components/facilities/manual-plan-upload-dialog.tsx:62`, `components/facilities/route-map.tsx:496`). `eslint.config.mjs` has no project rules beyond `next/core-web-vitals` + `next/typescript`.

Dead code:

- **Orphaned components (0 importers):** `components/facilities/schedule-card.tsx`, `components/facilities/vehicle-pickup-points-manager.tsx` (425 lines; sole caller of `getAddressesForPickupPoints`), `components/shared/mobile-nav.tsx` (notable — the spec requires mobile nav), `components/shared/page-header.tsx`, `profile-details-card.tsx`, `refresh-button.tsx`, `role-accent.tsx`, `stat-card.tsx`.

- **Uncalled exported Server Actions (still reachable endpoints):** `route-generation.actions.ts` `getPendingRoutes` (472), `getRouteDayStats` (506); `route-override.actions.ts` `reassignEmployeeToVehicle` (28), `swapEmployees` (239), `cancelRoute` (471), `getRouteDetails` (512); `route-setting.actions.ts` `setPickupPointActive` (126), `createVehicle` (272).
- **Unused lib exports:** `lib/routing/utils.ts` `chunkArray`, `clusterByDistance`, `roundRobinAssignment`, `addMinutesToTime`, `parseArrivalTimeShift`, `getDepartureTimeForShift`, `calculateCompletionTime`; `lib/routing/osrm.ts` `optimizeRoute`, `nearestNeighborOrder`, `pickupTimesFromLegs`, `calculatePickupTimes`; `lib/routing/geocoder.ts:97` `geocodeSearch`; `lib/constants/route-shifts.ts` `shiftKeyForArrivalTime`, `PICKUP_SHIFT`; `route-breakdown/hooks/useVehicleGroups.ts:100` `flattenTrips`.
- **Deprecated, 0 importers:** `lib/mailer.ts:39-40` `EMAIL_ENABLED`; `lib/constants/notification-recipients.ts:21-26` `FACILITIES_/OPERATIONS_NOTIFICATION_EMAIL`.
- **Unused/redundant deps:** `@dnd-kit/sortable`, `autoprefixer` (Tailwind v4 uses `@tailwindcss/postcss` only); `@neondatabase/serverless` is only transitively needed via `@prisma/adapter-neon`. `date-fns` pinned to v2 and used in 2 files.
- **Work-in-progress, unwired:** `lib/routing/corridor-core.ts`, `lib/queries/fleet.ts`, `lib/cache-tags.ts`.
- 1 TODO (`actions/user-management.actions.ts:66` test-mode flag). No commented-out code blocks.

Testing: 2 files / 19 `node:test` cases (`lib/routing/__tests__/side-selection.test.ts`, `manual-plan-training.test.ts`); **no `test` script**, no CI wiring, no vitest/playwright. None of the CLAUDE.md-required tests exist (deadline enforcement, role denial, address-change flow, audit log). Core engine (`planCorridorRoutes`, `computePickupSchedule`, `buildRetimeUpdates`, OSRM fallbacks, `loadLearnedLegs` aliasing) is untested.

Docs sprawl: 15 root `.md` files (~6,200 lines; `SESSION_MEMORY.md` alone ~2,300). Several are one-off (`GITHUB_CONNECTED.md`, `SETUP_COMPLETE.md`, `GIT_COMMANDS_REFERENCE.md`, `START_HERE.md`, `DOCUMENTATION_INDEX.md`); 8 still mention SQLite. `CLAUDE.md` §§2-9 describe the retired FastAPI/React stack, and its drift note points to directories no longer in the repo.

Where and How to Improve

- **Get ESLint to zero errors.** For the 6 `set-state-in-effect` errors (all "close dialog / reset form when action state changes"): derive state instead of syncing it, or move the reset into the action's success handler / `onSubmit` callback, or key the form on a `version` returned by the action. For `reset-password/[token]/page.tsx:17`, compute `now` once outside render logic (e.g. `const now = new Date()` passed into a pure `isTokenValid(user, now)` helper) or move validation into a `lib/queries` function. Fix the 9 warnings (remove unused props/vars, correct `useMemo` deps). Add `npm run lint` to CI as a gate.
- **Tighten the compiler:** enable `noUncheckedIndexedAccess`, `noUnusedLocals`, `noUnusedParameters`, `noImplicitReturns`, `noFallthroughCasesInSwitch`; set `target: "ES2022"`; drop `allowJs` if no JS sources remain. Replace `osrm.ts:328` `null as unknown as DistanceElement` with a proper `DistanceElement | null` type; validate the JWT payload with zod instead of the cast at `jwt.ts:42`; replace `!` in `weekly-schedule-table.tsx` and `pickup-schedule.ts` with guarded lookups.
- **Add project ESLint rules:** `no-restricted-imports` (prisma in pages/components; I/O in `*-core.ts`), `@typescript-eslint/consistent-type-imports`, `@typescript-eslint/no-non-null-assertion` (warn), `max-lines` (warn at 400) and `max-lines-per-function` (warn at 150). Replace `<img>` with `next/image` where possible to retire the 5 disables.

- **Delete dead code** (git history preserves it): the 8 orphan components (or wire `mobile-nav.tsx` into the layouts, since the spec requires it), the 8 uncalled Server Actions, the dead `utils.ts` / `osrm.ts` / `geocoder.ts` helpers, deprecated constants, `@dnd-kit/sortable`, `autoprefixer`. Add `knip` (`npx knip`) to CI to keep it clean.
- **Wire tests:** add `"test": "node --import tsx --test \"lib/**/__tests__/*.test.ts\""` and include it in `verify`. Priority tests: `getSubmissionWindow` deadline (`lib/date/week.ts:56`, Saturday 12:00 Asia/Dhaka edges); `requireRole` denial + proxy role-prefix logic; `assertCanManageTarget` (after S2); address-change approval flow; `audit()` creation; golden tests for `planCorridorRoutes` (old vs core); `computePickupSchedule`; `buildRetimeUpdates`. Consider vitest for component/action tests with a Prisma test DB (Neon branch).
- **Consolidate docs:** move durable docs into `docs/` (`ARCHITECTURE.md`, `RUNBOOK.md`, `DATABASE.md`, `PRD.md`), archive one-off setup notes to `docs/archive/`, trim `SESSION_MEMORY.md` to current state + open threads, and rewrite `CLAUDE.md` §§2-9 for the Next.js/Prisma stack so agents stop reading stale FastAPI instructions.
- Resolve the `isProd()` TODO in `user-management.actions.ts:66` (see S11).

Appendix A — Suggested remediation roadmap

Phase	Items	Effort
P0 — this week (security)	S1 (DTOs + Prisma `omit` for `passwordHash`/reset fields), S2 (user-management RBAC), S3 (hashed tokens, `APP_URL`, 30-min TTL, atomic consume), S5 (revoke sessions), delete uncalled Server Actions	~2–3 days
P1 — reliability	OSRM/Nominatim timeouts + limiter; remove render-time write in route-setting page; fix `reports.ts` in-memory date filter; `after()` for `weekly-schedule:295`; learning-loop status filter + transaction; ESLint errors → 0	~2–3 days
P2 — compliance	`AuditLog` model + `audit()` helper across all mutations; zod schemas per action (`withAction` wrapper); rate limiting; HTML escaping in emails; address-change approval flow	~1 week
P3 — performance	Wire `corridor-core` (parallel leg prefetch + sync plan); N+1 removal in 10 PM path / day-sheet save; matrix dedupe + chunking; cache layer with `updateTag`; FK index migration + drop redundant indexes; `Promise.all` page loads; bounded list queries	~1–2 weeks
P4 — maintainability	`lib/queries` boundary + ESLint enforcement; `RoleShell` + shared role pages; split mega-components; `*-core.ts` extraction; `routing/constants.ts`; Asia/Dhaka zoned-time helpers; tsconfig flags; test script + priority tests; docs consolidation	ongoing

Appendix B — Method

- Static review of all files in scope by four parallel review passes (security, routing design, performance, architecture/quality), each finding verified against source; headline findings (S1, S2, S3, render-time write, unwired `corridor-core`, no fetch timeouts, missing cache invalidation, `reports.ts` date filter) were re-verified directly.
- Tooling: `npx tsc --noEmit` (0 errors), `npx eslint .` (7 errors / 9 warnings), ripgrep scans for escape hatches, importer graphs for dead-code detection, read-only `git diff` / `git show` to compare

`algorithms.ts` with HEAD.

- Line numbers refer to the working tree on 2026-10-01 and will drift as code changes.